

City College of New York
Mechanical Engineering Department
ME 59917: Robotics
Professors Bo Wang and Inigo Sanz Pena

Analysis of a 6 Degree of Freedom Robot

By: Tahmidul Islam and Nabidul Islam

Due: May 21, 2026

Introduction

Overview

The robot selected for this project was a 6 degree of freedom (D.O.F.) anthropomorphic arm with a spherical wrist. The robot mirrors human arm motion and is used when precise positional and orientational movement is required. A common place an anthropomorphic arm can be found is in factories, where similar robots work on the conveyor line picking and placing objects onto the moving track, as well as hospitals, conducting precise surgeries. In most industrial cases, as well as this report, a 6 DOF anthropomorphic arm is considered to only have revolute joints, resulting in marginally smoother motion and a simpler system. In the context of this report, the robot is assumed to be inside a conceptual factory working on the conveyor line. The task of this robot is to remove objects from the conveyor belt while avoiding singularity. This report presents detailed analysis into configuring and deploying this task for the robot.

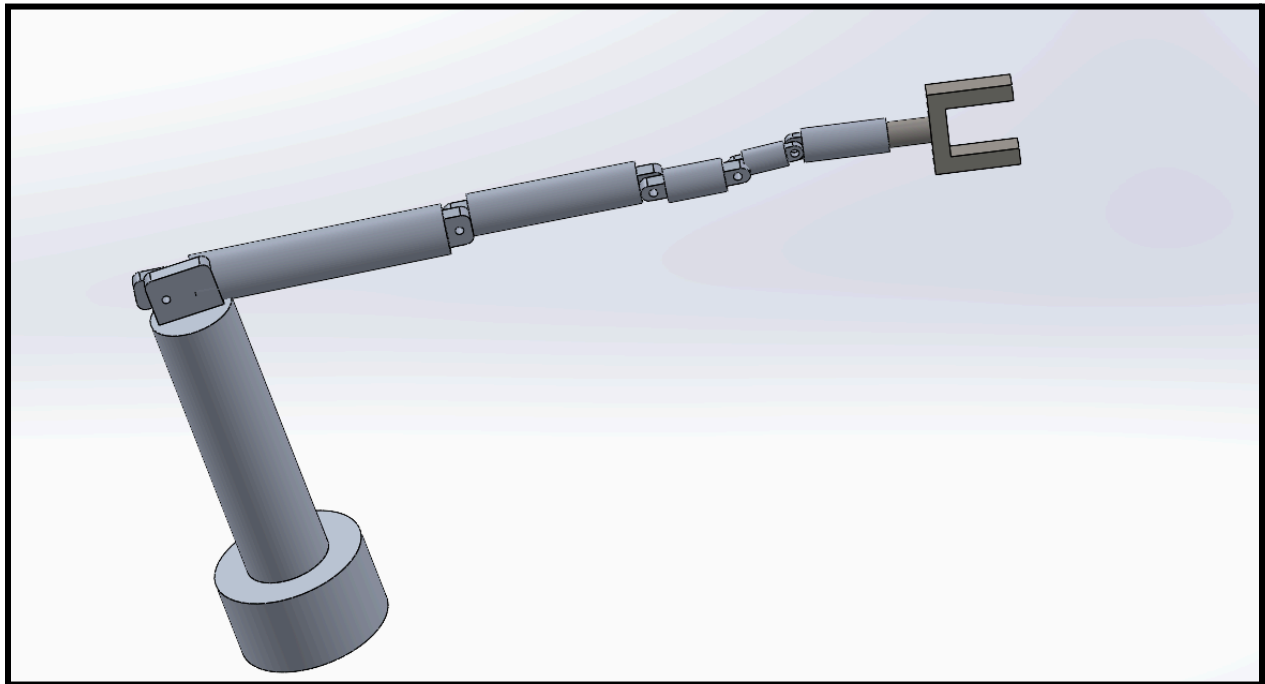
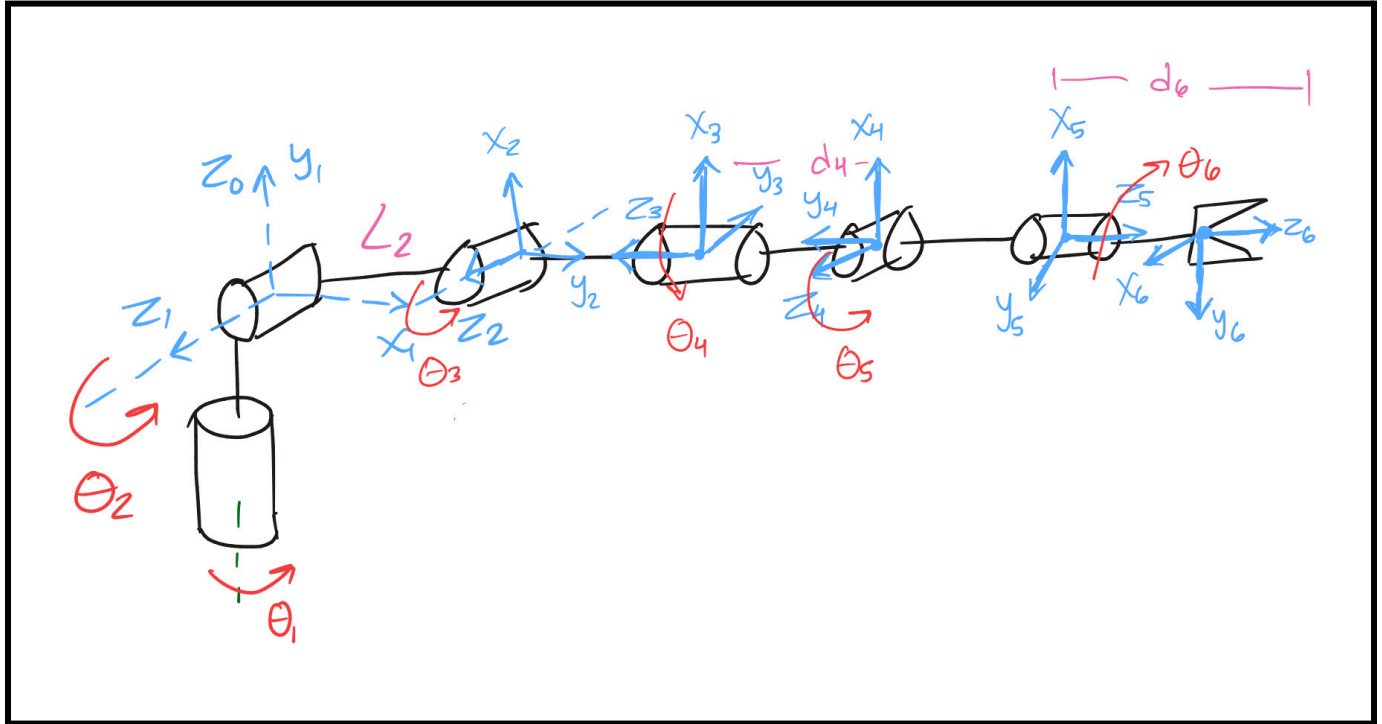


Figure 1: Rough C.A.D. Model of the 6 DOF Anthropomorphic Arm with a Spherical Wrist.

Forward Kinematics

Reference Frames



DH Table

i	d_i	θ_i	a_i	α_i
1	0	θ_1	0	$\pi/2$
2	0	θ_2	L_2	0
3	0	θ_3	0	$\pi/2$
4	d_4	θ_4	0	$-\pi/2$
5	0	θ_5	0	$\pi/2$
6	d_6	θ_6	0	0

DH Matrices

$$A_1 = \begin{bmatrix} C_1 & 0 & S_1 & 0 \\ S_1 & 0 & -C_1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} A_2 = \begin{bmatrix} C_2 & -S_2 & 0 & L_2 C_2 \\ S_2 & C_2 & 0 & L_2 S_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} A_3 = \begin{bmatrix} C_3 & 0 & S_3 & 0 \\ S_3 & 0 & -C_3 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_4 = \begin{bmatrix} C_4 & 0 & -S_4 & 0 \\ S_4 & 0 & C_4 & 0 \\ 0 & -1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} A_5 = \begin{bmatrix} C_5 & 0 & S_5 & 0 \\ S_5 & 0 & -C_5 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} A_6 = \begin{bmatrix} C_6 & -S_6 & 0 & 0 \\ S_6 & C_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

End-Effector Matrix

$$T_6^0 = A_1 A_2 A_3 A_4 A_5 A_6$$

$$T_6^0 = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$C_{23} = \cos(\theta_2 + \theta_3), \quad S_{23} = \sin(\theta_2 + \theta_3)$$

$$r_{11} = C_6 [-C_1 S_5 S_{23} + C_5 (C_1 C_4 C_{23} + S_1 S_4)] + S_6 [-C_1 S_4 C_{23} + C_4 S_1]$$

$$r_{12} = C_6 [-C_1 S_4 C_{23} + C_4 S_1] - S_6 [-C_1 S_5 S_{23} + C_5 (C_1 C_4 C_{23} + S_1 S_4)]$$

$$r_{13} = C_1 C_5 S_{23} + S_5 (C_1 C_4 C_{23} + S_1 S_4)$$

$$p_x = L_2 C_1 C_2 + d_4 C_1 S_{23} + d_6 r_{13}$$

$$r_{21} = -C_6 [C_5 (C_1 S_4 - C_4 S_1 C_{23}) + S_1 S_5 S_{23}] - S_6 (C_1 C_4 + S_1 S_4 C_{23})$$

$$r_{22} = -C_6 (C_1 C_4 + S_1 S_4 C_{23}) + S_6 [C_5 (C_1 S_4 - C_4 S_1 C_{23}) + S_1 S_5 S_{23}]$$

$$r_{23} = C_5 S_1 S_{23} - S_5 (C_1 S_4 - C_4 S_1 C_{23})$$

$$p_y = L_2 S_1 C_2 + d_4 S_1 S_{23} + d_6 r_{23}$$

$$r_{31} = C_6 (C_4 C_5 S_{23} + S_5 C_{23}) - S_4 S_6 S_{23}$$

$$r_{32} = -C_6 S_4 S_{23} - S_6 (C_4 C_5 S_{23} + S_5 C_{23})$$

$$r_{33} = C_4 S_5 S_{23} - C_5 C_{23}$$

$$p_z = L_2 S_2 - d_4 C_{23} + d_6 r_{33}$$

Inverse Kinematics

General Approach and Solution

The inverse kinematics was used to find the joint angles needed for the robot to place the end effector at a desired position and orientation. Since this robot has a spherical wrist, the problem can be separated into two parts. The first part is the position problem, which is used to solve joints 1, 2, and 3. The second part is the orientation problem, which solves joints 4, 5, and 6.

This separation is helpful since the first three joints mostly decide where the wrist center goes. The last three joints mostly decide how the gripper is rotated. Therefore, the inverse kinematics was done in the following way:

Position IK: solve $\theta_1, \theta_2, \theta_3$

Orientation IK: solve $\theta_4, \theta_5, \theta_6$

The desired end-effector pose is written as

$$T_0^6 = \begin{bmatrix} R_0^6 & o_T \\ 0 & 1 \end{bmatrix}$$

where o_T is the desired tool-center position and R_0^6 is the desired rotation matrix of the end effector.

$$o_T = \begin{bmatrix} x_T \\ y_T \\ z_T \end{bmatrix}$$

$$R_0^6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

The way the tool will reach the box on the table is outlined by the third column of the rotation matrix:

$$a_T = \begin{bmatrix} r_{13} \\ r_{23} \\ r_{33} \end{bmatrix}$$

Since the tool center is offset from the wrist center by the length d_6 , the wrist center is found by subtracting the tool offset:

$$o_c = o_T - d_6 a_T$$

or

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} x_T \\ y_T \\ z_T \end{bmatrix} - d_6 \begin{bmatrix} r_{13} \\ r_{23} \\ r_{33} \end{bmatrix}$$

Once the wrist center is determined, the first joint angle can be found from the horizontal position of the wrist center:

$$\theta_1 = \text{atan2}(y_c, x_c)$$

The radial distance from the base axis is

$$r = \sqrt{x_c^2 + y_c^2}$$

and the vertical distance is

$$z = z_c$$

The distance from the shoulder joint to the wrist center is

$$\rho = \sqrt{r^2 + z^2}$$

Using the law of cosines, the elbow term is

$$D = \frac{r^2 + z^2 - L_2^2 - d_4^2}{2L_2d_4}$$

The elbow angle can then be found using

$$\delta = \pm \cos^{-1}(D)$$

The positive and negative signs represent the two arm positions, elbow-up and elbow-down. After this, the third joint angle is found from

$$\theta_3 = \delta + \frac{\pi}{2}$$

The second joint angle is found using the arm geometry which is:

$$\theta_2 = \text{atan2}(z, r) - \text{atan2}(d_4 \sin \delta, L_2 + d_4 \cos \delta)$$

After θ_1 , θ_2 , and θ_3 are found, the rotation from frame 0 to frame 3 is found using the first three transformations:

$$R_0^3 = R_0^3(\theta_1, \theta_2, \theta_3)$$

Then the wrist rotation is found using

$$R_3^6 = (R_0^3)^T R_0^6$$

This equation shows that the rotation still has to be created by the wrist. Once the arm moves the wrist center to its accurate location, the other joints need to move to match the orientation.

Let

$$R_3^6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

For the spherical wrist, the wrist angles are determined using the following equations

$$s_5 = \sqrt{r_{13}^2 + r_{23}^2}$$

$$\theta_5 = \text{atan2}(s_5, r_{33})$$

$$\theta_4 = \text{atan2}(r_{23}, r_{13})$$

$$\theta_6 = \text{atan2}(r_{32}, -r_{31})$$

A second solution can be found for these joint angles by assuming that the wrist is flipped:

$$\theta_5 = \text{atan2}(-s_5, r_{33})$$

$$\theta_4 = \text{atan2}(-r_{23}, -r_{13})$$

$$\theta_6 = \text{atan2}(-r_{32}, r_{31})$$

If s_5 is approximately 0, the wrist is near a singularity. In which case, joints 4 and 6 are no longer independent, so one of them can be set to zero and the other one can be solved from the remaining rotation.

For this robot to complete its task, the desired tool-center position was said to be

$$o_T = \begin{bmatrix} 1.000 \\ 0.300 \\ 0.150 \end{bmatrix} \text{ m}$$

with dimensions assumed to be

$$L_2 = 0.60 \text{ m}, \quad d_4 = 0.55 \text{ m}, \quad d_6 = 0.25 \text{ m}$$

The desired end-effector orientation to complete the task was said to be

$$R_0^6 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

The approach vector is represented by the third column of the rotation matrix:

$$a_T = \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}$$

Using that and the robot's dimension, the wrist center can be calculated as follows

$$o_c = o_T - d_6 a_T$$

$$o_c = \begin{bmatrix} 1.000 \\ 0.300 \\ 0.150 \end{bmatrix} - 0.25 \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}$$

$$o_c = \begin{bmatrix} 1.000 \\ 0.300 \\ 0.400 \end{bmatrix} \text{ m}$$

From these calculations,

$$x_c = 1.000, \quad y_c = 0.300, \quad z_c = 0.400$$

The first joint angle is

$$\theta_1 = \text{atan2}(0.300, 1.000) = 16.699^\circ$$

The radial distance is

$$r = \sqrt{1.000^2 + 0.300^2} = 1.044 \text{ m}$$

The distance to the wrist center is

$$\rho = \sqrt{1.044^2 + 0.400^2} = 1.118 \text{ m}$$

Since

$$L_2 + d_4 = 0.60 + 0.55 = 1.15 \text{ m}$$

and

$$\rho = 1.118 \text{ m} < 1.15 \text{ m}$$

which shows that the point is inside the robot workspace. This means the desired target can be reached by the robot.

Then, the law-of-cosines term is

$$D = \frac{\rho^2 - L_2^2 - d_4^2}{2L_2d_4}$$

$$D = \frac{1.118^2 - 0.60^2 - 0.55^2}{2(0.60)(0.55)} = 0.8902$$

Resulting in,

$$\delta = \cos^{-1}(0.8902) = 27.108^\circ$$

For this solution, the negative elbow branch was used:

$$\delta = -27.108^\circ$$

The third joint angle is

$$\theta_3 = \delta + 90^\circ = -27.108^\circ + 90^\circ = 62.892^\circ$$

The second joint angle is

$$\theta_2 = \text{atan2}(0.400, 1.044) - \text{atan2}(0.55\sin(-27.108^\circ), 0.60 + 0.55\cos(-27.108^\circ))$$

$$\theta_2 = 20.970^\circ - (-12.947^\circ) = 33.917^\circ$$

After the first three joints determine the placement of the wrist center, the wrist angles are solved using

$$R_3^6 = (R_0^3)^T R_0^6$$

Solving for the remaining three joint angles results in

$$\theta_4 = 0^\circ, \quad \theta_5 = -96.809^\circ, \quad \theta_6 = 16.699^\circ$$

The final inverse kinematics solution is

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \end{bmatrix} = \begin{bmatrix} 16.699^\circ \\ 33.917^\circ \\ 62.892^\circ \\ 0^\circ \\ -96.809^\circ \\ 16.699^\circ \end{bmatrix}$$

This configuration places the wrist center accurately at the desired location and gives the end effector the orientation required to reach and grab the object. The joint angles were checked by plugging back into the forward kinematic equations.

Jacobian Matrix and Velocity Kinematics

Jacobian Matrix

In order to calculate the jacobian matrix, first the DH table was utilized in order to create the homogeneous transformation matrix (A_i). The homogeneous transformation matrix formula used is provided below.

$$A_i = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In the matrix A_i , c represents cosine while s represents sine. Using this equation, all the homogeneous transformation matrices were derived for all parameters from the DH table. These matrices were kept in variable form to allow easier substitution later on during the analysis. These matrices are provided below.

$$\begin{aligned} A_1 &= \begin{bmatrix} c\theta_1 & 0 & s\theta_1 & 0 \\ s\theta_1 & 0 & -c\theta_1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & A_4 &= \begin{bmatrix} c\theta_4 & 0 & -s\theta_4 & 0 \\ s\theta_4 & 0 & c\theta_4 & 0 \\ 0 & -1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ A_2 &= \begin{bmatrix} c\theta_2 & -s\theta_2 & 0 & L_2 c\theta_2 \\ s\theta_2 & c\theta_2 & 0 & L_2 s\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & A_5 &= \begin{bmatrix} c\theta_5 & 0 & s\theta_5 & 0 \\ s\theta_5 & 0 & -c\theta_5 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ A_3 &= \begin{bmatrix} c\theta_3 & 0 & s\theta_3 & 0 \\ s\theta_3 & 0 & -c\theta_3 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & A_6 &= \begin{bmatrix} c\theta_6 & -s\theta_6 & 0 & 0 \\ s\theta_6 & c\theta_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

After calculating the homogeneous transformation matrix for the individual links, they were multiplied together using the formula provided below.

$$T_n^0 = A_1 \cdot A_2 \cdot \dots \cdot A_n$$

This was done in order to have the homogeneous transformation matrix of one link with respect to another, allowing proper definition of the position and orientation of the end effector with respect to all the links in the robot. Some of the calculated matrices are provided below.

$$T_{01} = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{02} = \begin{bmatrix} c_1 c_2 & -c_1 s_2 & s_1 & c_1 c_2 L_2 \\ c_2 s_1 & -s_1 s_2 & -c_1 & c_2 L_2 s_1 \\ s_2 & c_2 & 0 & L_2 s_2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{03} = \begin{bmatrix} c_1 c_2 c_3 - c_1 s_2 s_3 & s_1 & c_1 c_2 s_3 + c_1 c_3 s_2 & c_1 c_2 L_2 \\ c_2 c_3 s_1 - s_1 s_2 s_3 & -c_1 & c_2 s_1 s_3 + c_3 s_1 s_2 & c_2 L_2 s_1 \\ c_2 s_3 + c_3 s_2 & 0 & s_2 s_3 - c_2 c_3 & L_2 s_2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{04} = \begin{bmatrix} c_4(c_1 c_2 c_3 - c_1 s_2 s_3) + s_1 s_4 & -c_1 c_3 s_2 - c_1 c_2 s_3 & c_4 s_1 - s_4(c_1 c_2 c_3 - c_1 s_2 s_3) & c_1 c_2 L_2 + d_4(c_1 c_2 s_3 + c_1 c_3 s_2) \\ -c_1 s_4 + c_4(c_2 c_3 s_1 - s_1 s_2 s_3) & -c_3 s_1 s_2 - c_2 s_1 s_3 & -s_4(c_2 c_3 s_1 - s_1 s_2 s_3) - c_1 c_4 & c_2 L_2 s_1 + d_4(c_2 s_1 s_3 + c_3 s_1 s_2) \\ c_4(c_2 s_3 + c_3 s_2) & -(s_2 s_3 - c_2 c_3) & -s_4(c_2 s_3 + c_3 s_2) & L_2 s_2 + d_4(s_2 s_3 - c_2 c_3) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

T_{05} and T_{06} were also calculated. However, the matrices were too large for the final symbolic value to be shown. Due to the size of the matrices, $\cos(\theta_n)$ or $\sin(\theta_n)$ was simplified into c_n or s_n .

Since this robot has only revolute joints, the formula for revolute joint assembly of the jacobian was utilized. This formula is provided below.

$$J_{vi} = z_{i-1}(o_n^0 - o_{i-1})$$

$$J_{\omega i} = z_{i-1}$$

J_v represents linear velocity and J_w represents angular velocity. z_i and o_i are both found from the corresponding combined homogeneous transformation matrices (T_i). An example is shown below.

$$T_{01} = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The green box represents z_1 and the red box represent o_i . This was done for all combined homogeneous transformation matrices.

Finally, the jacobian matrix is assembled using the formula provided below.

$$J(q) = \begin{bmatrix} J_v \\ J_w \end{bmatrix}$$

Since the robot has 6 degrees of freedom, $J(q)$ is a 6x6 matrix. As a result, the symbolic matrix is too large to include in the results. However, the jacobian matrix at the desired end effector position is shown below.

The desired End Effector Positions:

$$\begin{aligned} x &= 1m \\ y &= 0.3m \\ z &= 0.15m \end{aligned}$$

With dimensions:

$$L_2 = 0.6m \quad d_4 = 0.55m \quad d_6 = 0.25m$$

Jacobian Matrix at the desired End Effector Position.

$$J(q) = \begin{bmatrix} -0.3000 & -0.1437 & 0.1770 & -0.0713 & 0.2395 & 0 \\ 1.0000 & -0.0431 & 0.0531 & 0.2378 & 0.0718 & 0 \\ 0 & 1.0440 & 0.5461 & 0 & 0 & 0 \\ 0 & 0.2873 & 0.2873 & 0.9511 & 0.2873 & 0 \\ 0 & -0.9578 & -0.9578 & 0.2853 & -0.9578 & 0 \\ 1.0000 & 0 & 0 & 0.1186 & 0 & -1.0000 \end{bmatrix}$$

Time Derivative

In order to find the time derivative of the jacobian matrix, the chain rule and partial differentiation was utilized. Since the the jacobian matrix is built using joint angles ($q = \theta$), it is the joint angles the matrix is derived with respect to. This is done for all the joint angles in the robot. The formula for the time derivative of the jacobian matrix is provided below.

$$\dot{J} = \frac{\partial J}{\partial q_1} \dot{q}_1 + \frac{\partial J}{\partial q_2} \dot{q}_2 + \cdots + \frac{\partial J}{\partial q_6} \dot{q}_6$$

Since the time derivative matrix is also a 6x6 matrix, it is too large to include the symbolic result. However, the time derivative matrix at the desired end effector position is shown below.

The desired End Effector Positions:

$$\begin{aligned} x &= 1m \\ y &= 0.3m \\ z &= 0.15m \end{aligned}$$

With dimensions:

$$L_2 = 0.6m \quad d_4 = 0.55m \quad d_6 = 0.25m$$

Time Derivative Matrix:

$$\dot{J}(q) = \begin{bmatrix} -0.0050 & 0.0073 & 0.0005 & -0.0009 & -0.0003 & 0 \\ 0.0033 & 0.0016 & 0.0009 & -0.0002 & 0.0009 & 0 \\ 0 & 0.0046 & 0.0001 & -0.0000 & 0.0000 & 0 \\ 0 & 0.0035 & 0.0035 & -0.0009 & 0.0035 & 0.0000 \\ 0 & 0.0011 & 0.0011 & 0.0035 & 0.0011 & 0.0000 \\ 0 & 0 & 0 & -0.0014 & 0.0000 & 0 \end{bmatrix}$$

Inverse Velocity

In order to calculate inverse velocity, first the determinant of the jacobian matrix has to be calculated. This has to be the first step because if $\det(J) \neq 0$ then direct inversion is possible. Since this is a square matrix, it is possible to calculate the determinant. However, since the jacobian matrix is a 6x6 matrix, the symbolic determinant is too large to include. Results for the determinant of the jacobian matrix at the desired end effector position are presented below instead.

The desired End Effector Positions:

$$\begin{aligned}x &= 1m \\y &= 0.3m \\z &= 0.15m\end{aligned}$$

With dimensions:

$$L_2 = 0.6m \quad d_4 = 0.55m \quad d_6 = 0.25m$$

Jacobian Determinant:

$$0.1559$$

Therefore, since $\det(J) \neq 0$ has been confirmed, direct inversion of the jacobian matrix is applicable in this situation.

In order to calculate the inverse velocity, the formula provided below was used.

$$\dot{q} = J^{-1}\dot{x}$$

Provided below is a plot of the inverse velocity of all the joints over the runtime of the robot.

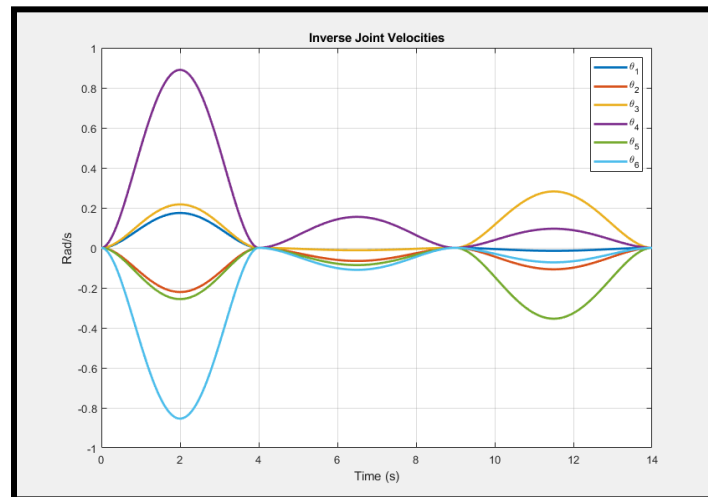


Figure A: Plot of Inverse Joint Velocities

Inverse Acceleration

For inverse acceleration, the inverse velocity equation is differentiated to get the equation below.

$$\ddot{q} = J^{-1} (\ddot{x} - \dot{J}\dot{q})$$

Provided below is a plot of the inverse acceleration of all the joints over the runtime of the robot.

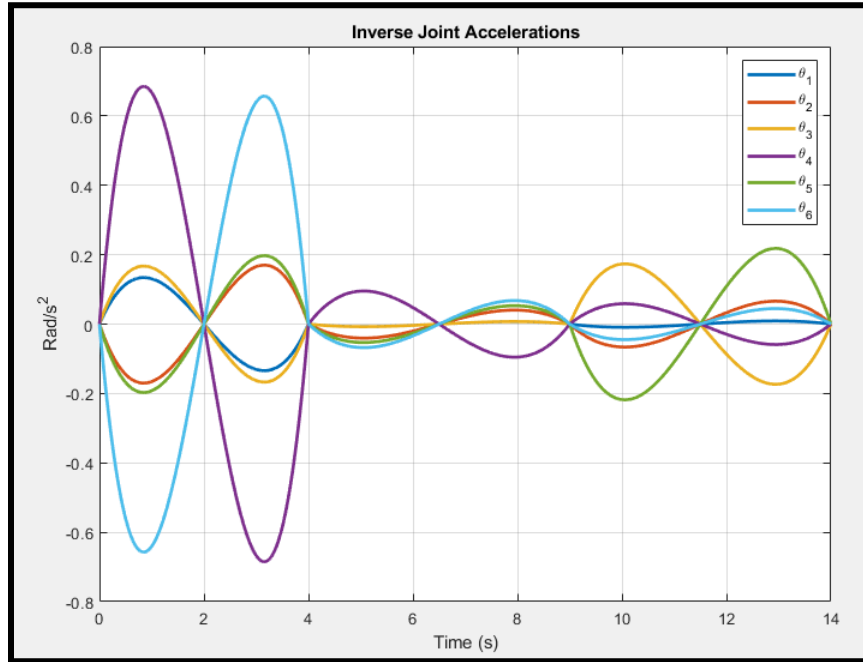


Figure B: Plot of Inverse Joint Accelerations

Path and Trajectory

Trajectory and Path Setup and Data

In order to set up a correct trajectory and path, start and end points are required. For this robot, the start and end points of the end effector are as defined:

Robot Start Point (X, Y, Z,) m: (0.8, 0, 0.6)

Robot End Point (X, Y, Z,) m: (1, 0.3, 0.15)

Another critical aspect that needs to be defined is the initial and target orientation of the end effector. For this robot, the initial and target orientation of the end effector are as defined:

Robot Initial Orientation:

$$R_0 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Robot Goal Orientation:

$$R_{\text{goal}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

The initial orientation of the robot was found by reading the 3x3 matrix on the left in the combined T_6^0 homogenous transformation matrix by substituting the initial joint position values.

The initial joint position values can be found below.

$$q_0 = \begin{bmatrix} 0 \\ \frac{\pi}{2} \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

In order to properly do the task assigned, the path of the robot from start to end point is split up into 4 pieces. Each piece is defined below, along with segment configuration and run times.

Piece 1: Home Position (X,Y,Z) m
(0.8, 0, 0.6)

Segment 1: Piece 1 → Piece 2
Time Duration: 4 seconds

Piece 2: Pre-Grabbing Position (X,Y,Z) m
(0.8, 0, 0.6) → (1, 0.3, 0.35)

Segment 2: Piece 2 → Piece 3
Time Duration: 5 seconds

Piece 3: Grabbing Position (X,Y,Z) m
(1, 0.3, 0.35) → (1, 0.3, 0)

Segment 3: Piece 3 → Piece 4
Time Duration: 4.5 seconds

Piece 4 Post-Grabbing Position (X,Y,Z) m:
(1, 0.3, 0) → (1, 0.3, 0.15)

Due to their being four separate pieces, interpolation between each is required to fill in the gaps created between the points and to allow a smooth trajectory. This generates the required position and orientation data used in the rest of the trajectory configuration. The equations used for position and orientation interpolation are provided below respectively.

$$P(t) = (1 - a)p_n + ap_{n+1}$$

$$R(t) = R_0(R_0^T R_{goal})^s$$

$$R_{rel} = R_0^T R_{goal}$$

In the equations above, a and s are moving from 0 to 1.

Once interpolation of position and rotation creates points, the script starts at the initial joint configuration and computes the forward kinematics to evaluate the location and orientation of the end effector at this current configuration. The current location and orientation is then compared to the desired location and orientation and an error is calculated using the formula below.

$$\|p(q) - p_{des}\|^2 + \|\log(R_{des}^T R(q))\|^2$$

The right hand side of the equation calculates orientation errors and the left hand side calculates position error.

From this total error, a corrected motion is created that moves the end effector closer to the goal position and target. This is $\dot{x}$. Using the jacobian matrix relationship shown below, the joint velocity of the new corrected motion is calculated.

$$\dot{x} = J(q)\dot{q}$$

For the case of this robot, the starting joint configuration is very close to reaching singularity. Therefore using the standard J^{-1} function causes issues with the resulting trajectories. As a result, the formula below was utilized in order to find the joint velocities.

$$\dot{q} = J^T (JJ^T + \lambda^2 I)^{-1} \dot{x}$$

This helps avoid any issues near the singularity.

Finally, this new joint velocity is saved and updated into the path using the formula below. This entire process will now loop and the next joint angle analyzed will be closer to the goal position and orientation.

$$q_{k+1} = q_k + \dot{q} dt$$

Once all the joint angles have been calculated, they will be imputed into the quintic polynomial trajectory formula provided below in order to level out the path and produce smoother position, velocity and acceleration graphs.

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

Robot movement in space is calculated using the formulas provided below.

$$\dot{x} = J(q)\dot{q}$$

$$\ddot{x} = J(q)\ddot{q} + \dot{J}(q)\dot{q}$$

In order to make sure the robot was moving properly singularity and manipulability checks were done utilizing the formulas below.

$$\text{Singularity Check: } \det(J) = 0$$

$$\text{Manipulability Check: } \sqrt{\det(J \cdot J^T)}$$

Finally, inverse velocity and accelerations were also plotted in order to confirm that the plots all match.

Trajectory Equations

Below is the general piecewise format of the quintic polynomial trajectory equations.

$$q_i(t) = \begin{cases} a_{i0}^{(1)} + a_{i1}^{(1)}t + a_{i2}^{(1)}t^2 + a_{i3}^{(1)}t^3 + a_{i4}^{(1)}t^4 + a_{i5}^{(1)}t^5, & 0 \leq t < 4 \\ a_{i0}^{(2)} + a_{i1}^{(2)}(t-4) + a_{i2}^{(2)}(t-4)^2 + a_{i3}^{(2)}(t-4)^3 + a_{i4}^{(2)}(t-4)^4 + a_{i5}^{(2)}(t-4)^5, & 4 \leq t < 9 \\ a_{i0}^{(3)} + a_{i1}^{(3)}(t-9) + a_{i2}^{(3)}(t-9)^2 + a_{i3}^{(3)}(t-9)^3 + a_{i4}^{(3)}(t-9)^4 + a_{i5}^{(3)}(t-9)^5, & 9 \leq t \leq 13.5 \end{cases}$$

Below is the table of the values of all the coefficients in the quintic polynomial trajectory equations.

Joint	Segment	a_0	a_1	a_2	a_3	a_4	a_5
1	1	-0.001539	0	0	0.058104	-0.021789	0.002179
1	2	0.370328	0	0	-0.002502	0.000751	-0.000060
1	3	0.339047	0	0	-0.004322	0.001441	-0.000128
2	1	1.565180	0	0	-0.076309	0.028616	-0.002862
2	2	1.076805	0	0	-0.019320	0.005796	-0.000464
2	3	0.835302	0	0	-0.016974	0.005658	-0.000503
3	1	0.010953	0	0	0.047112	-0.017667	0.001767
3	2	0.312469	0	0	-0.000740	0.000222	-0.000018
3	3	0.303223	0	0	0.067216	-0.022405	0.001992
4	1	-2.600882	0	0	0.256755	-0.096283	0.009628
4	2	-0.957652	0	0	0.045925	-0.013778	0.001102
4	3	-0.383585	0	0	0.038321	-0.012774	0.001135
5	1	0.006552	0	0	-0.066394	0.024898	-0.002490
5	2	-0.418371	0	0	-0.011287	0.003386	-0.000271
5	3	-0.559460	0	0	-0.101553	0.033851	-0.003009
6	1	2.601523	0	0	-0.239832	0.089937	-0.008994
6	2	1.066600	0	0	-0.036912	0.011074	-0.000886
6	3	0.605195	0	0	-0.033431	0.011144	-0.000991

Trajectory Results

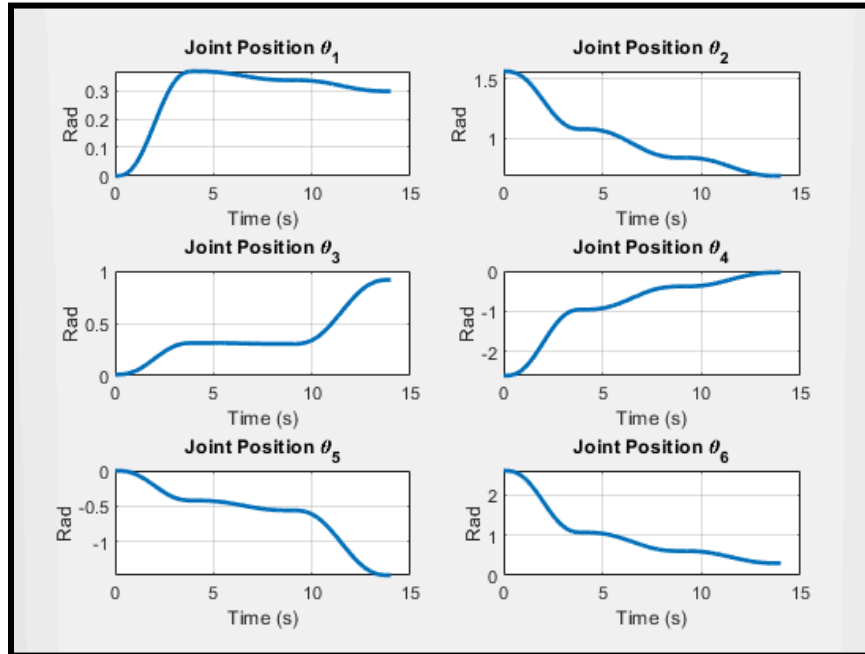


Figure C: Plot of Joint Position vs Time from Trajectory

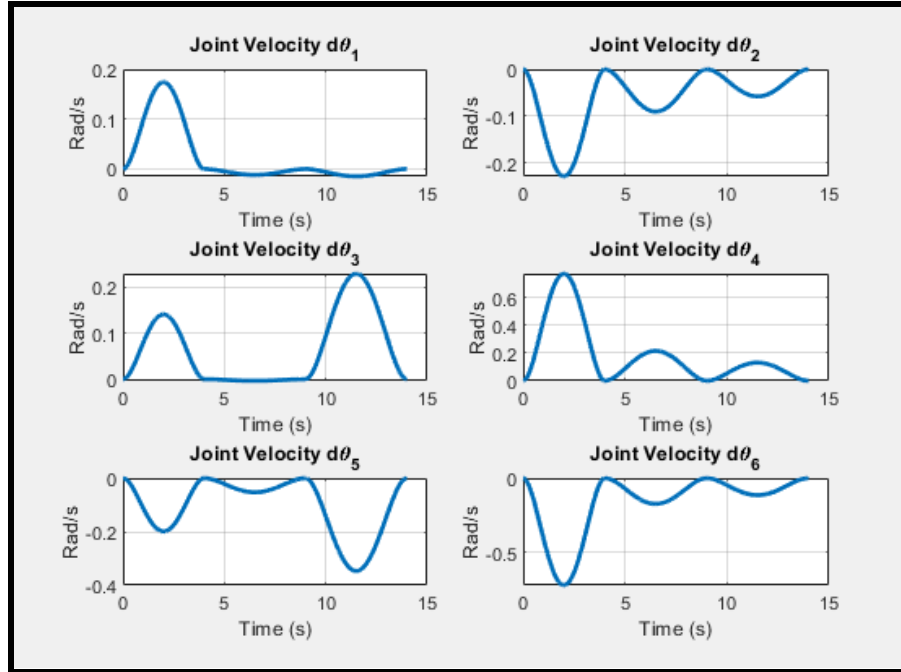


Figure D: Plot of Joint Velocity vs Time from Trajectory

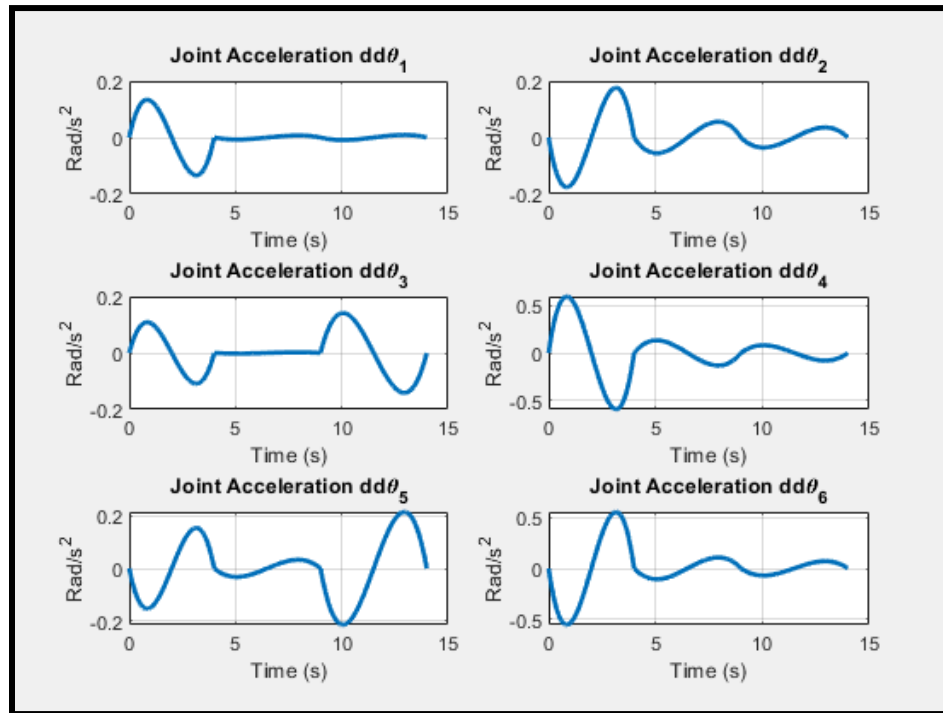


Figure E: Plot of Joint Acceleration vs Time from Trajectory

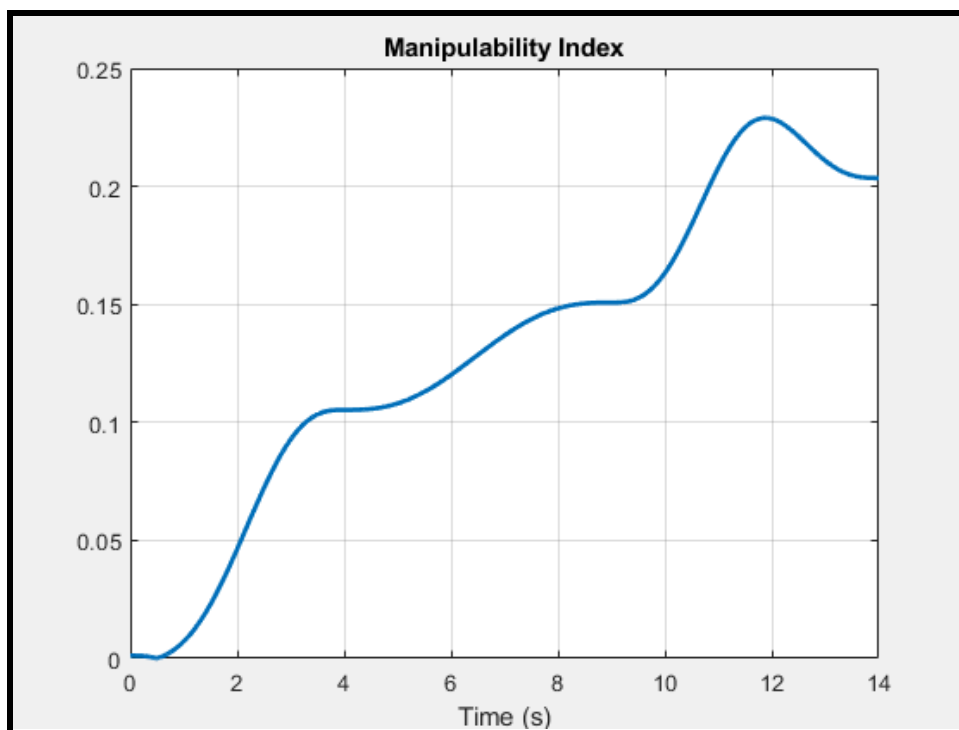


Figure F: Plot of Manipulability Values over Time.

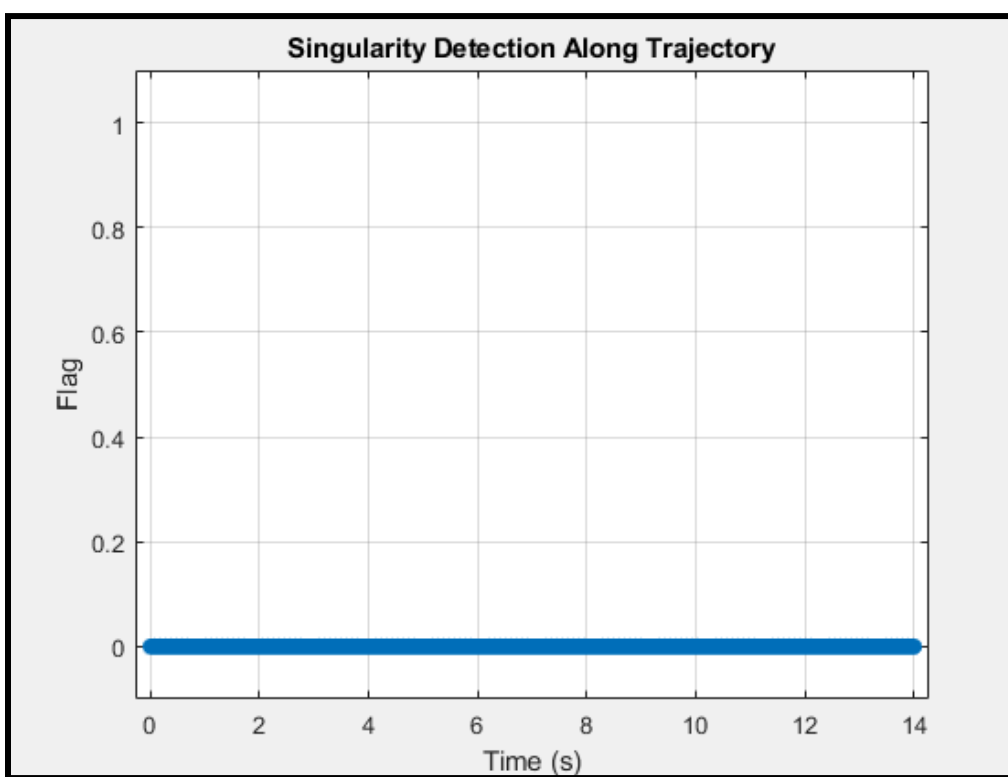


Figure G: Plot of Singularity Detection over Time (0 means no singularity detected. 1 means singularity detected).

Equations of Motion

Physical Description of Robot

This robot was designed as a 6-DOF anthropomorphic arm with a spherical wrist. The main dimensions used for the robot were

$$L_2 = 0.60 \text{ m}, \quad d_4 = 0.55 \text{ m}, \quad d_6 = 0.25 \text{ m}$$

where L_2 is the main arm length, d_4 is the forearm offset, and d_6 is the end-effector offset. These values enable the robot enough space to move outward, reach down to the object, and back up. To determine the equations of motion, the mass, center of mass, and inertia values were reasonably assumed for the six links. The assumed mass vector was:

$$m = \begin{bmatrix} 5.0 \\ 4.0 \\ 3.0 \\ 1.5 \\ 1.0 \\ 0.8 \end{bmatrix} \text{ kg}$$

The robot was designed with heavier links near the base and lighter links the closer you got to the end-effector. The first couple of links carry most of the structure, therefore are required to be sturdier and heavier. Furthermore, the center of mass for the larger links were located further from the joint, such as 0.225 m for link 2, whereas the COM for the smaller links were closer to the joints. The inertia matrix was also assumed to be diagonal.

Equations of Motion

To determine the equations of motion for this 6-DOF robot, the standard robot dynamic form can be used:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + B\dot{q}$$

Where q , $\dot{q}$, and $\ddot{q}$ represents the joint angle, velocity, and acceleration respectively, $M(q)$ is the inertia matrix, $C(q, \dot{q})\dot{q}$ is the Coriolis and centrifugal term, $G(q)$ is gravity, and $B\dot{q}$ is damping or friction.

The joint angle vector is represented as:

$$q(t) = \begin{bmatrix} \theta_1(t) \\ \theta_2(t) \\ \theta_3(t) \\ \theta_4(t) \\ \theta_5(t) \\ \theta_6(t) \end{bmatrix}$$

The velocity and acceleration vectors were represented as

$$\dot{q}(t) = \begin{bmatrix} \dot{\theta}_1(t) \\ \dot{\theta}_2(t) \\ \dot{\theta}_3(t) \\ \dot{\theta}_4(t) \\ \dot{\theta}_5(t) \\ \dot{\theta}_6(t) \end{bmatrix}, \quad \ddot{q}(t) = \begin{bmatrix} \ddot{\theta}_1(t) \\ \ddot{\theta}_2(t) \\ \ddot{\theta}_3(t) \\ \ddot{\theta}_4(t) \\ \ddot{\theta}_5(t) \\ \ddot{\theta}_6(t) \end{bmatrix}$$

where $M(q)$ is the inertia matrix, $C(q, \dot{q})\dot{q}$ is the Coriolis and centrifugal term, $G(q)$ is gravity, and $B\dot{q}$ is damping or friction.

For this robot, the equation of motion then becomes:

$$\begin{bmatrix} \tau_1 \\ \tau_2 \\ \tau_3 \\ \tau_4 \\ \tau_5 \\ \tau_6 \end{bmatrix} = M(q) \begin{bmatrix} \ddot{\theta}_1 \\ \ddot{\theta}_2 \\ \ddot{\theta}_3 \\ \ddot{\theta}_4 \\ \ddot{\theta}_5 \\ \ddot{\theta}_6 \end{bmatrix} + C(q, \dot{q})\dot{q} + G(q) + B\dot{q}$$

Utilizing MATLAB, the equation of motion is rearranged to solve for the joint accelerations with the general equation becoming:

$$\ddot{q} = M^{-1}(\tau - B\dot{q} - d)$$

The acceleration was then integrated over time. The velocity function became

$$\dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t$$

and the position function became

$$q_{k+1} = q_k + \dot{q}_{k+1} \Delta t$$

where

$$\Delta t = 0.005 \text{ s}$$

This means that for each time step, the controller calculates the torque, the plant model uses that torque to calculate acceleration, and then the velocity and position are updated. Then by using the MATLAB, and inputting our robot dimensions, joint angles and other parameters, the following results were determined.

$$M(q) = \begin{bmatrix} 6.055 & 0.0230 & -0.0129 & -0.1636 & 0.0676 & -0.0971 \\ 0.0230 & 8.8331 & 2.7082 & 0.0834 & 0.2945 & 0.0621 \\ -0.0129 & 2.7082 & 1.6437 & 0.0613 & 0.1926 & 0.0520 \\ -0.1636 & 0.0834 & 0.0613 & 0.0320 & 0.0126 & 0.0108 \\ 0.0676 & 0.2945 & 0.1926 & 0.0126 & 0.0727 & 0.0112 \\ -0.0971 & 0.0621 & 0.0520 & 0.0108 & 0.0112 & 0.0175 \end{bmatrix}$$

$$C(q, \dot{q})\dot{q} = \begin{bmatrix} 0.0005637 \\ -0.0001671 \\ -0.0002565 \\ -0.0000233 \\ -0.0000184 \\ -0.0000176 \end{bmatrix}$$

$$G(q) = \begin{bmatrix} 0 \\ 71.0872 \\ 22.0975 \\ 0.7831 \\ 2.3290 \\ 0.5494 \end{bmatrix}$$

$$B\dot{q} = \begin{bmatrix} 0.0112 \\ -0.0425 \\ 0.0258 \\ -0.0200 \\ 0.0023 \\ 0.0113 \end{bmatrix}$$

Therefore, the torque vector is determined to be

$$\tau = \begin{bmatrix} 0.0181 \\ 71.0298 \\ 22.1193 \\ 0.7627 \\ 2.3310 \\ 0.5005 \end{bmatrix} \text{ N} \cdot \text{m}$$

The diagonal values of the mass/inertia matrix are representative of what occurs at the joints. The larger values occur at Joint 1 and 2 showing just how hard it is to accelerate each joint. Joint 2 has a large value, expectedly since it acts as a shoulder joint and moves a big portion of the robot arm. The Coriolis and centrifugal vector is very small, at almost zero compared to the gravity vector. This indicates that joint velocity did not play a crucial role in the motion of the robot which is to be expected since the robot moves slowly to reach, grasp, and pick up the object. Gravity is the main contributor to torque with Joint 2 and 3 carrying most of the load. This is expected since those joints support most of the weight during the reaching motion. Damping was also relatively small compared to gravity, indicating that it has some effect on the motors but not a lot.

Control Laws

The PID and PD controllers are used to make the robot follow a desired trajectory to complete tasks. The robot was not just sent straight to the final joint angles but rather, it followed a connected trajectory with multiple points along the path. This is a more realistic approach since the robot has to start from the home position, move toward the object, and then finish at the target pose.

The desired trajectory was written as

$$q_d(t) = \begin{bmatrix} \theta_{1,d}(t) \\ \theta_{2,d}(t) \\ \theta_{3,d}(t) \\ \theta_{4,d}(t) \\ \theta_{5,d}(t) \\ \theta_{6,d}(t) \end{bmatrix}$$

with desired joint velocity

$$\dot{q}_d(t) = \begin{bmatrix} \dot{\theta}_{1,d}(t) \\ \dot{\theta}_{2,d}(t) \\ \dot{\theta}_{3,d}(t) \\ \dot{\theta}_{4,d}(t) \\ \dot{\theta}_{5,d}(t) \\ \dot{\theta}_{6,d}(t) \end{bmatrix}$$

The trajectory was made using four waypoints from the planned path. These points were connected using MATLAB's quintic polynomial trajectory. This add-on was used since it produces smooth position, velocity, and acceleration. This is relatively important since sharp changes in motion can cause sudden torque changes and make the robot's motion unrealistic.

The joint tracking error was defined as

$$e(t) = q_d(t) - q(t)$$

where $q(t)$ is the actual joint position. The velocity error was determined using

$$\dot{e}(t) = \dot{q}_d(t) - \dot{q}(t)$$

The controller uses these errors to calculate the motor torque input:

$$\tau(t) = \begin{bmatrix} \tau_1(t) \\ \tau_2(t) \\ \tau_3(t) \\ \tau_4(t) \\ \tau_5(t) \\ \tau_6(t) \end{bmatrix}$$

Formulation of PD

The PD controller uses proportional and derivative control. The control law can be described with the following equation:

$$\tau_{PD}(t) = K_p e(t) + K_d \dot{e}(t)$$

Removing the tracking errors results in

$$\tau_{PD}(t) = K_p (q_d(t) - q(t)) + K_d (\dot{q}_d(t) - \dot{q}(t))$$

For each joint, the equation is transformed into

$$\tau_i(t) = K_{pi}(\theta_{i,d}(t) - \theta_i(t)) + K_{di}(\dot{\theta}_{i,d}(t) - \dot{\theta}_i(t))$$

The proportional part of the controller moves the joint toward the desired joint angle. When the position error becomes really large, the controller applies more torque. The derivative part of the controller uses the velocity error to help reduce overshoot and helps stop the joint from moving too fast compared to the desired trajectory.

The gain matrices are diagonal:

$$K_p = \begin{bmatrix} K_{p1} & 0 & 0 & 0 & 0 & 0 \\ 0 & K_{p2} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{p3} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{p4} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{p5} & 0 \\ 0 & 0 & 0 & 0 & 0 & K_{p6} \end{bmatrix}$$

$$K_d = \begin{bmatrix} K_{d1} & 0 & 0 & 0 & 0 & 0 \\ 0 & K_{d2} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{d3} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{d4} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{d5} & 0 \\ 0 & 0 & 0 & 0 & 0 & K_{d6} \end{bmatrix}$$

The gains were determined based on a second-order approximation of each joint. For a second-order system,

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$$

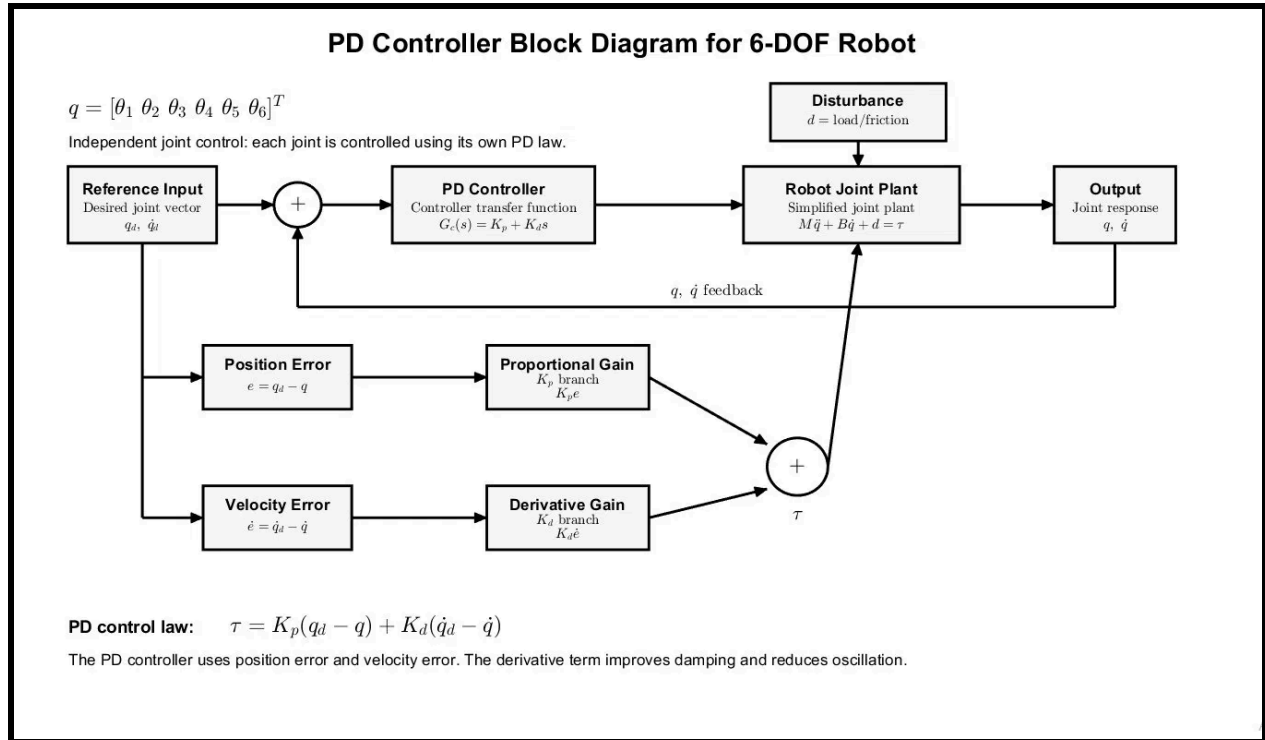
The proportional gain can be found using the equation:

$$K_p = M\omega_n^2$$

and the derivative gain was determined using:

$$K_d = 2\zeta\omega_n M - B$$

where M is the effective inertia, B is damping, ω_n is the natural frequency, and ζ is the damping ratio. Increasing ω_n makes the response faster, while the damping ratio controls how much oscillation is reduced.



Formulation of PID

The PID controller builds on the PD controller by adding an integral term to it. The control law is:

$$\tau_{PID}(t) = K_p e(t) + K_i \int e(t) dt + K_d \dot{e}(t)$$

Substituting the tracking errors gives

$$\tau_{PID}(t) = K_p (q_d(t) - q(t)) + K_i \int (q_d(t) - q(t)) dt + K_d (\dot{q}_d(t) - \dot{q}(t))$$

For each joint, the equation is transformed into:

$$\tau_i(t) = K_{pi} (\theta_{i,d}(t) - \theta_i(t)) + K_{ii} \int (\theta_{i,d}(t) - \theta_i(t)) dt + K_{di} (\dot{\theta}_{i,d}(t) - \dot{\theta}_i(t))$$

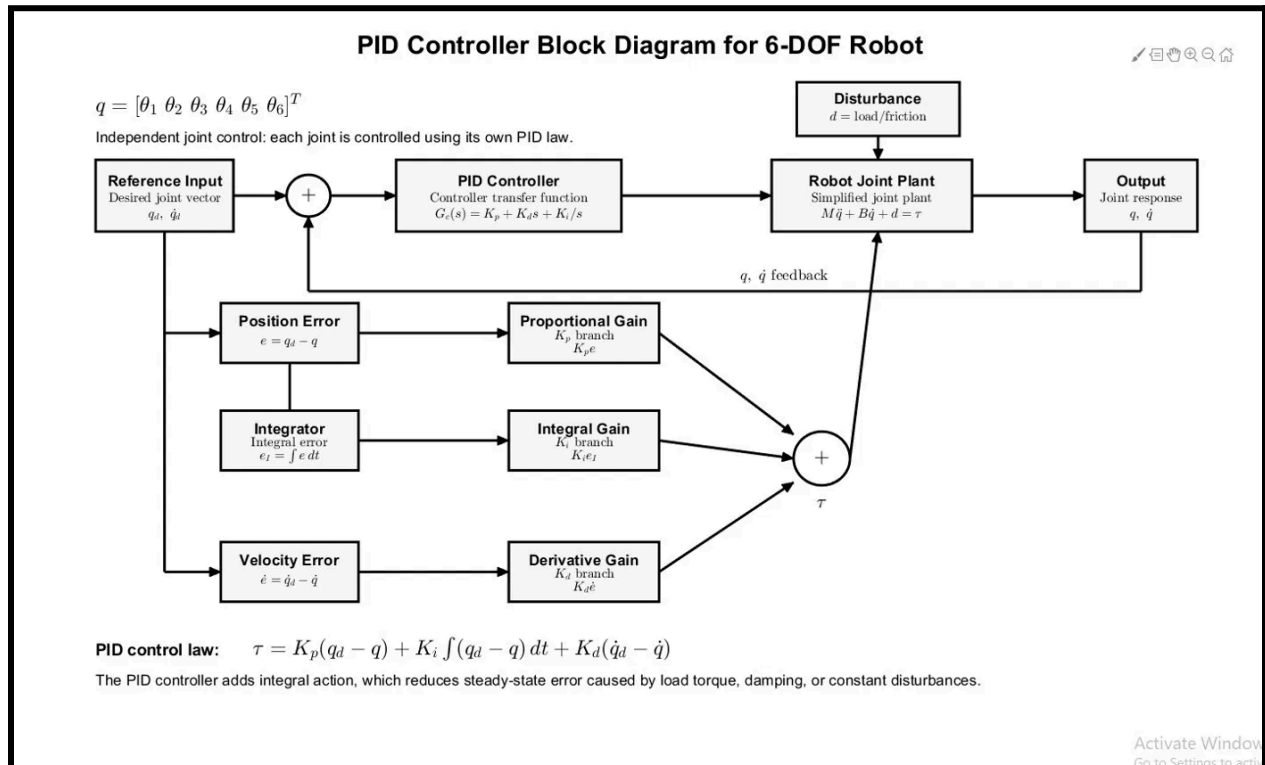
The integral error is determined using:

$$e_{int}(t) = \int_0^t e(\tau) d\tau$$

In the numerical code, it was changed to

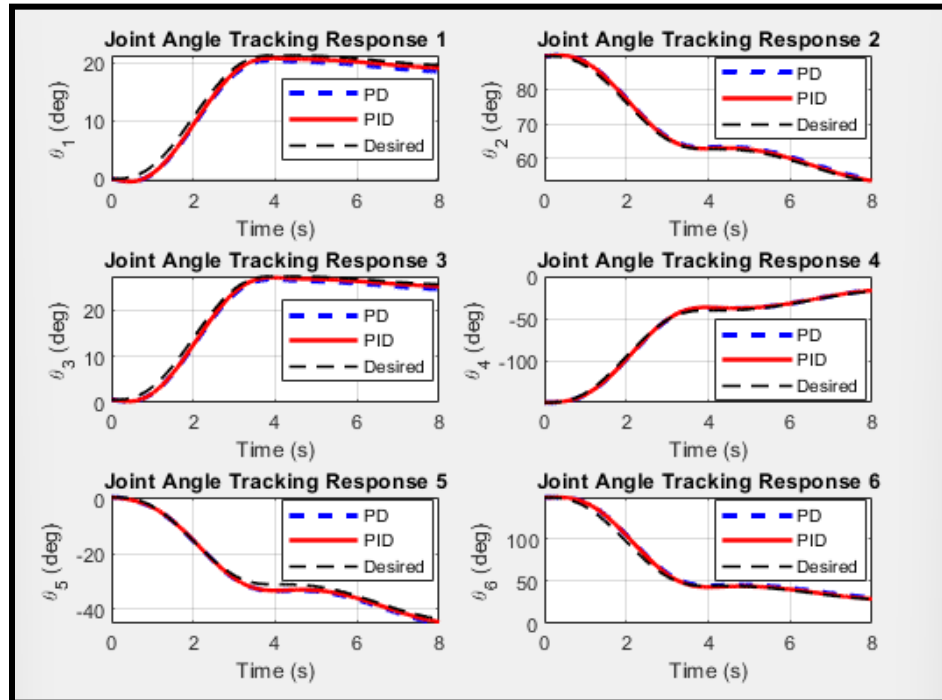
$$e_{int,k} = e_{int,k-1} + e_k \Delta t$$

The integral term was added since it helps remove steady-state error from the system. If the robot has a small offset because of the presence of disturbance torque, friction, or modeling error, a PD controller may not remove all of it. The integral term will keep adding the error over time and pushes the response closer to the desired value. However, too much integral control can also cause overshoot or lead to very large torque. Because of this, the integral term was limited to avoid this.

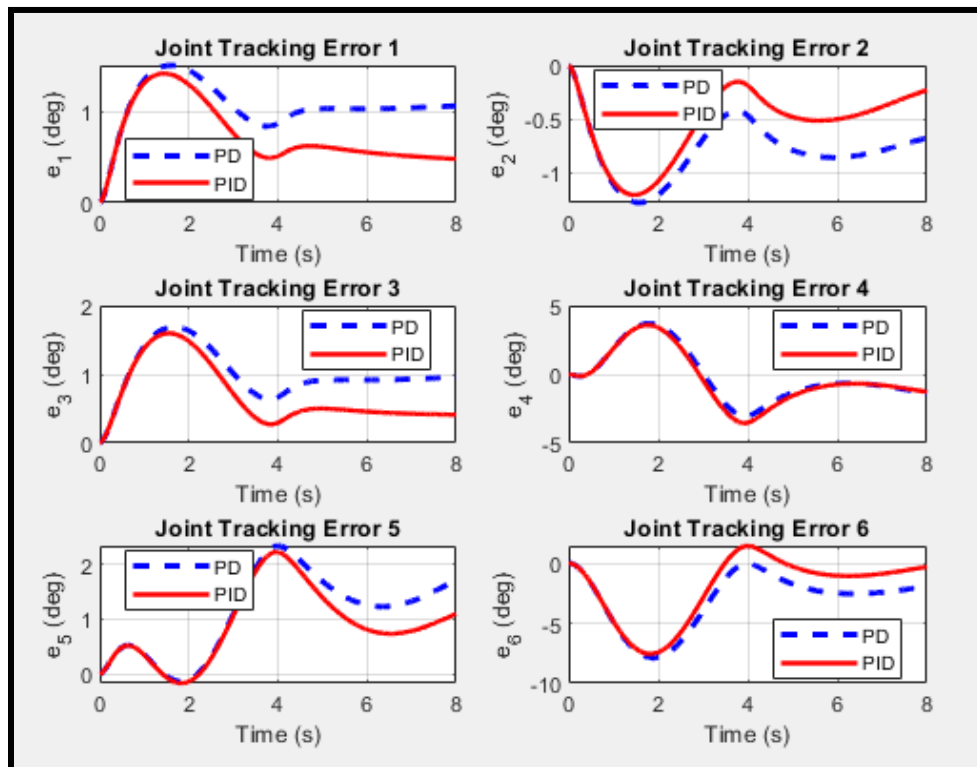


Control Law Results

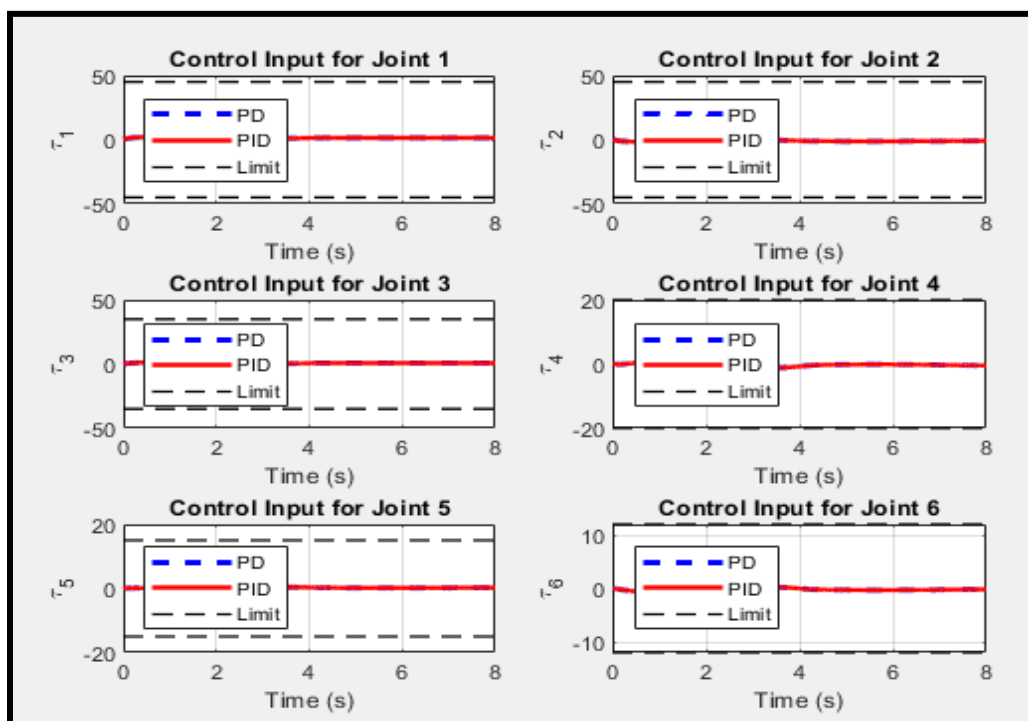
Path Plots



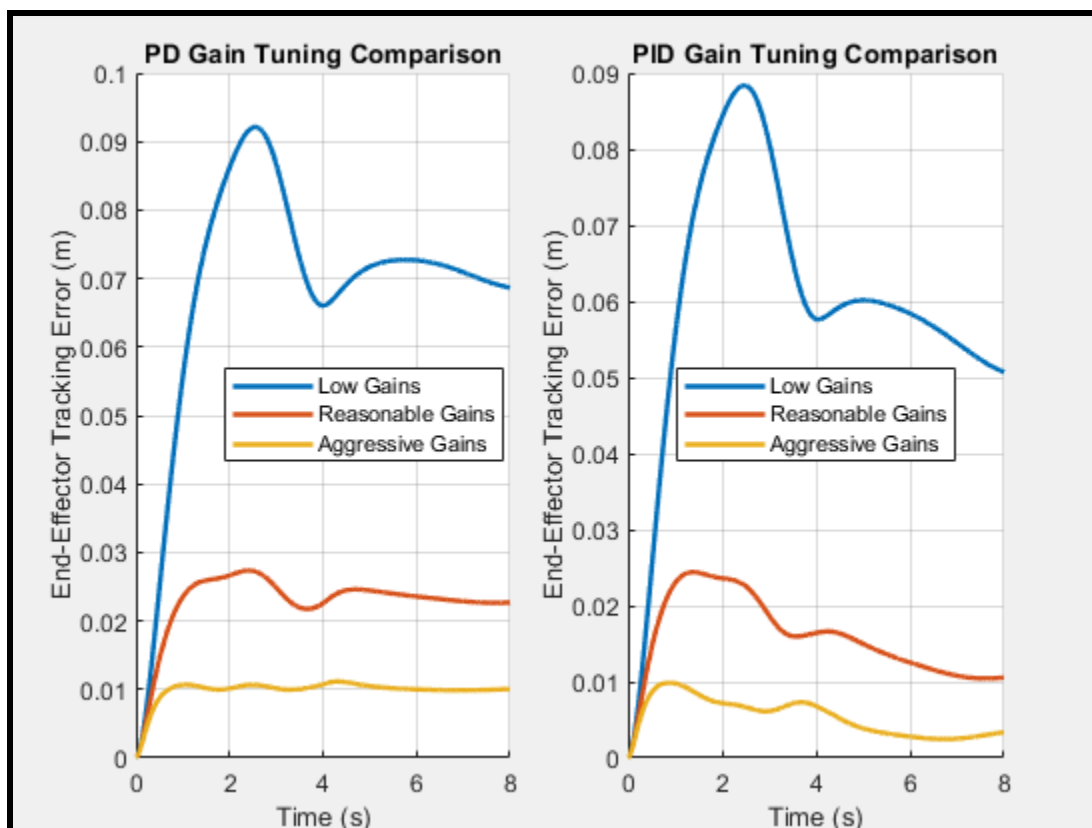
Position Error Plots

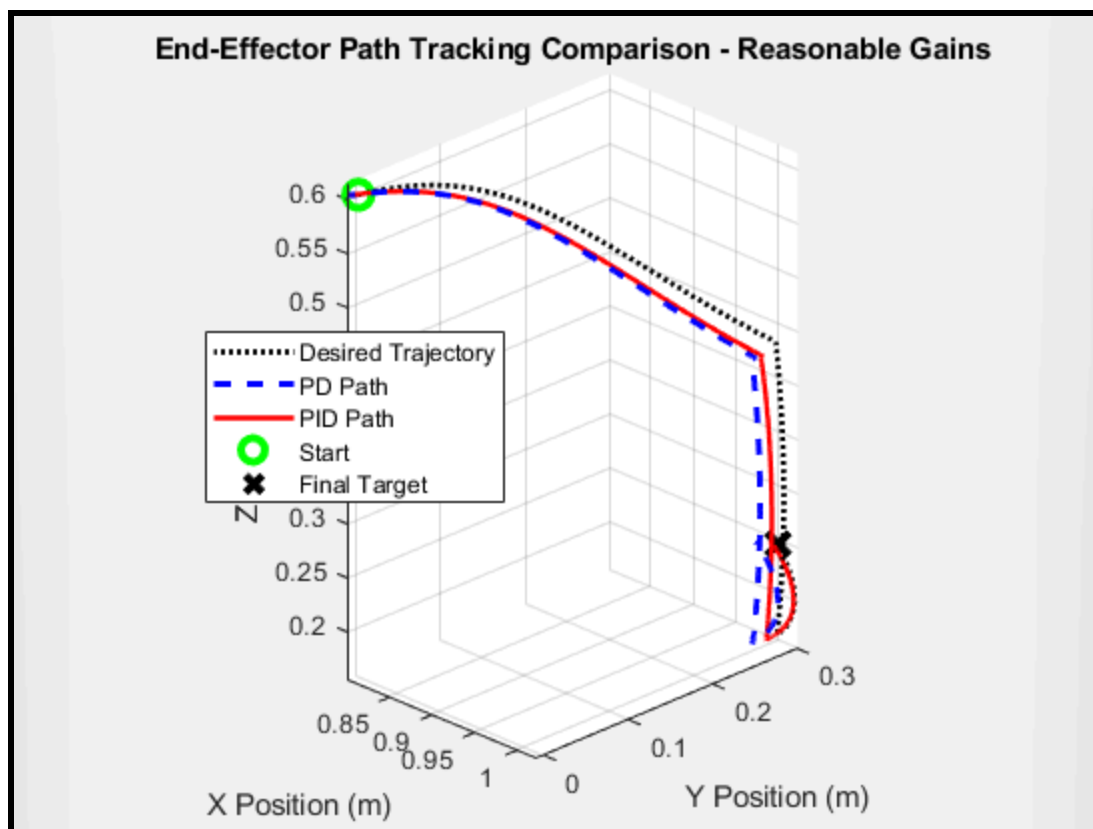


Motor Torque Input



Gain Comparison





Control Law Comparison

Comparison of PD and PID

All in all, both controllers were able to successfully move the robot through the determined trajectory and complete the reach, grasp, and pick up task. The PD controller was stable and kept the motor torque within the desired limits. However, it had more tracking error during the robot's motion and ended up with a larger end-effector error.

The PID controller performed better since it stayed closer to the desired path and ended with a smaller position error. The main reason this happened was because of the integral term which helped remove the error that the PD controller is unable to fully correct.

The torque plots also show that the PID controller did not exceed the torque motors desired limits. This means the improved tracking was still adequate for the assumed motors. For this specific scenario, the PID controller is the better choice since the robot needs to be able to accurately reach the object's location. A smaller end-effector error gives the gripper a higher than likely chance of reaching the correct position. The PD controller is a simpler and stable controller, but the PID controller gives better accuracy for this robot and its task.

Conclusion

Final Remarks

All in all, this robot is versatile and able to accomplish the task that was set without running into major problems and successfully able to avoid singularities. A huge problem that was encountered while analyzing this robot was creating the path and trajectory. Because the path had four individual pieces that combined to form into one large path, there was four times the computation power and time required to run the code. Furthermore, because one piece depended heavily on the other, one faulty segment path generation rendered the entire trajectory useless. In the future, the robots starting configuration should not allow most of the joints to stack into each other. This would allow an easier path and trajectory planning.

References

“Inverse Kinematics in Robotics Using Neural Networks.” *ScienceDirect*, Elsevier, <https://www.sciencedirect.com/science/article/pii/S0263224120313154>. Accessed 21 May 2026.

“SLInKi: State Lattice Based Inverse Kinematics—A Fast, Accurate, and Flexible IK Solver for Soft Continuum Robot Manipulators.” *NASA Technical Reports Server (NTRS)*, NASA, <https://ntrs.nasa.gov/api/citations/19950005142/downloads/19950005142.pdf>. Accessed 21 May 2026.

Jin, Wanxin. “Lecture 9: Inverse Kinematics.” *Arizona State University Robotics Lecture Notes*, <https://wanxinjin.github.io/asu-robotics/lec9/ik.html>. Accessed 21 May 2026.

“Anthropomorphic Arm with Spherical Wrist Configuration and Dedicated Link Frames.” *ResearchGate*, https://www.researchgate.net/figure/Anthropomorphic-arm-with-spherical-wrist-configuration-and-dedicated-link-frames_fig2_276159627. Accessed 21 May 2026.

“Inverse Kinematic for 6DOF Arm.” *MATLAB Central File Exchange*, MathWorks, <https://www.mathworks.com/matlabcentral/fileexchange/30243-inverse-kinematic-for-6dof-arm>. Accessed 21 May 2026.

“Lecture 6: Inverse Kinematics Survey.” *Carnegie Mellon University*, <https://www.cs.cmu.edu/~15464-s13/lectures/lecture6/iksurvey.pdf>. Accessed 21 May 2026.

Lynch, Kevin M., and Frank C. Park. *Inverse Kinematics*. Carnegie Mellon University, 2013, <https://www.cs.cmu.edu/~15464-s13/lectures/lecture6/iksurvey.pdf>. Accessed 21 May 2026.